PostgreSQL con Docker en ProyectoCrediya

Guía paso a paso desde cero (Windows) para reemplazar H2 por PostgreSQL en el microservicio crediya-autenticacion-ms

Qué vas a lograr

El proyecto es un microservicio Spring Boot con Clean Architecture (scaffold de Bancolombia). Hoy guarda los datos en **H2 en memoria**: cada vez que la app se reinicia, todo se borra. Al terminar esta guía tendrás:

- Docker Desktop, Git y Java 25 instalados y verificados.
- Un contenedor **PostgreSQL 16** corriendo en tu máquina con datos persistentes.
- El proyecto configurado para conectarse a Postgres en lugar de H2.
- Tus cambios en una rama *feature*, subidos a GitHub y fusionados con Pull Requests.

Resumen de los archivos que se tocan

Archivo	Acción
docker-compose.yml (raíz)	Crear: define el contenedor de Postgres
infrastructure/driven-adapters/jpa-repository/build.gradle	Editar: driver de Postgres, H2 solo para tests
applications/app-service/src/main/resources/application.yaml	Editar: URL, usuario, contraseña y dialecto

> **Tip:** todos los comandos de esta guía se escriben en **PowerShell** (clic derecho en Inicio → Terminal). Copia y pega cada bloque tal cual.

1. Instalar las herramientas

1.1 WSL 2 (requisito de Docker en Windows)

Abre PowerShell **como administrador** y ejecuta. Al terminar, reinicia el PC.

```
wsl --install
```

1.2 Git, Java 25 y Docker Desktop

Con **winget** (viene incluido en Windows 10/11). Puedes ejecutarlos en la misma terminal de administrador:

```
winget install --id Git.Git -e
winget install --id EclipseAdoptium.Temurin.25.JDK -e
winget install --id Docker.DockerDesktop -e
```

El proyecto usa **Java 25** (está en gradle.properties como `javaVersion=25`), así que necesitas exactamente esa versión del JDK.

> **Ojo:** si winget no encuentra algún paquete, descárgalo manualmente: Git en git-scm.com, Java 25 (Temurin) en adoptium.net y Docker Desktop en docker.com/products/docker-desktop.

1.3 Abrir Docker Desktop

Abre Docker Desktop desde el menú Inicio, acepta los términos y espera a que abajo a la izquierda diga **Engine running**. Docker tiene que estar abierto siempre que trabajes con la base de datos.

1.4 Verificar que todo quedó instalado

Cierra y vuelve a abrir PowerShell (normal, no administrador) y ejecuta:

```
git --version
java -version
docker --version
docker compose version
```

Cada comando debe mostrar una versión. Si `java -version` no muestra 25, revisa la variable de entorno JAVA_HOME (ver sección de problemas).

1.5 Configurar tu identidad en Git (solo la primera vez)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@ejemplo.com"
```

2. Clonar el proyecto y crear tu rama

Pide al dueño del repositorio que te agregue como colaborador en GitHub (Settings → Collaborators) para poder subir ramas. Luego:

```
cd $HOME
git clone https://github.com/juanmejia017/ProyectoCrediya.git
cd ProyectoCrediya
git checkout develop
git pull origin develop
git checkout -b feature/postgres-docker
```

A partir de aquí, todos los comandos se ejecutan dentro de la carpeta `ProyectoCrediya`. Abre la carpeta en tu editor (IntelliJ IDEA o VS Code) para editar los archivos.

3. Crear docker-compose.yml

Crea un archivo nuevo llamado `docker-compose.yml` en la **raíz del proyecto** (la misma carpeta donde está `build.gradle`) con este contenido:

ProyectoCrediya/docker-compose.yml

```
services:
  postgres:
    image: postgres:16
    container_name: crediya-postgres
    environment:
      POSTGRES_USER: crediya
      POSTGRES_PASSWORD: crediya
      POSTGRES_DB: crediya_db
    ports:
      - "5432:5432"
    volumes:
      - crediya_pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U crediya -d crediya_db"]
      interval: 5s
      timeout: 5s
      retries: 5
    restart: unless-stopped

volumes:
  crediya_pgdata:
```

Línea	Para qué sirve
image: postgres:16	Imagen oficial de PostgreSQL versión 16
POSTGRES_USER / PASSWORD / DB	Usuario, contraseña y base que se crean la primera vez
ports: "5432:5432"	Expone el puerto de Postgres en tu máquina (localhost:5432)
volumes: crediya_pgdata	Guarda los datos en un volumen: no se pierden al apagar el contenedor
healthcheck	Docker revisa cada 5 s que la base esté lista para recibir conexiones

> **Ojo:** YAML es sensible a la indentación: usa **espacios**, nunca tabulaciones.

4. Cambiar el driver de base de datos

Abre `infrastructure/driven-adapters/jpa-repository/build.gradle` y reemplaza todo el bloque `dependencies` por:

infrastructure/driven-adapters/jpa-repository/build.gradle

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.reactivecommons.utils:object-mapper-api:0.1.0'

    runtimeOnly 'org.postgresql:postgresql'

    testRuntimeOnly 'com.h2database:h2'
    testImplementation 'org.reactivecommons.utils:object-mapper:0.1.0'
}
```

¿Por qué H2 no se borra del todo? El test `JpaConfigTest` usa `org.h2.Driver`. Con `testRuntimeOnly` H2 solo existe al correr los tests, y la aplicación real usa Postgres.

5. Configurar la conexión en application.yaml

Abre `applications/app-service/src/main/resources/application.yaml` y reemplaza **todo** su contenido por:

applications/app-service/src/main/resources/application.yaml

```
server:
  port: 8080
spring:
  application:
    name: "crediya-autenticacion-ms"
  devtools:
    add-properties: false
  profiles:
    include: null
  datasource:
    url: "${DB_URL:jdbc:postgresql://localhost:5432/crediya_db}"
    username: "${DB_USERNAME:crediya}"
    password: "${DB_PASSWORD:crediya}"
    driverClassName: "org.postgresql.Driver"
  jpa:
    databasePlatform: "org.hibernate.dialect.PostgreSQLDialect"
management:
  endpoints:
    web:
      exposure:
        include: "health,prometheus"
  endpoint:
    health:
      probes:
        enabled: true
cors:
  allowed-origins: "http://localhost:4200,http://localhost:8080"
```

Qué cambió respecto al original:

- Se eliminó el bloque `h2.console` (la consola web de H2 ya no aplica).
- La URL ahora apunta a `localhost:5432/crediya_db`, el contenedor del paso 3.
- `${DB_URL:valor}` significa: usa la variable de entorno DB_URL si existe; si no, usa el valor por defecto. Así el mismo archivo sirve en local y en un servidor.
- Driver y dialecto cambian a los de PostgreSQL.

> **Ojo:** los valores de usuario, contraseña y base deben coincidir **exactamente** con los del docker-compose.yml.

6. Levantar PostgreSQL

Con Docker Desktop abierto, desde la raíz del proyecto:

```
docker compose up -d
```

La primera vez descarga la imagen (puede tardar un par de minutos). Verifica que esté corriendo:

```
docker ps
```

En la columna STATUS debe aparecer `Up ... (healthy)` para `crediya-postgres`. Para entrar a la consola de Postgres:

```
docker exec -it crediya-postgres psql -U crediya -d crediya_db
```

Comandos útiles dentro de psql:

Comando	Qué hace
\l	Lista las bases de datos
\dt	Lista las tablas de la base actual
\d nombre_tabla	Muestra las columnas de una tabla
\q	Sale de psql

7. Compilar y ejecutar el proyecto

```
.\gradlew.bat clean build
.\gradlew.bat :app-service:bootRun
```

Si arranca bien, verás en la consola que Hikari abrió el pool de conexiones y que Tomcat está en el puerto 8080. Comprueba el health check en el navegador:

```
http://localhost:8080/actuator/health
```

Para detener la aplicación: **Ctrl + C** en la terminal.

> **Ojo:** es posible que `bootRun` falle con "*Not a managed type: class java.lang.Object*". **No es un error de Postgres.** `JPARepository` y `JPARepositoryAdapter` todavía tienen `Object` como placeholder del scaffold y `User.java` está vacío. Se soluciona en el siguiente avance, creando la entidad `UserEntity` en el adaptador JPA. La conexión a Postgres igual queda verificada con los pasos 6 y 8.

8. Conectarte con DBeaver o pgAdmin (opcional)

Si prefieres ver la base con una interfaz gráfica, crea una conexión PostgreSQL con estos datos:

Campo	Valor
Host	localhost
Puerto	5432
Base de datos	crediya_db
Usuario	crediya
Contraseña	crediya
URL JDBC	jdbc:postgresql://localhost:5432/crediya_db

Instalar DBeaver (opcional)

```
winget install --id dbeaver.dbeaver -e
```

9. Subir los cambios y hacer los Pull Requests

Revisa qué cambió y haz el commit con formato *conventional commits*:

```
git status
git add docker-compose.yml
git add infrastructure/driven-adapters/jpa-repository/build.gradle
git add applications/app-service/src/main/resources/application.yaml
git commit -m "chore: configurar PostgreSQL con Docker Compose para entorno local"
git push -u origin feature/postgres-docker
```

Luego en GitHub (pestaña **Pull requests → New pull request**):

- **PR 1:** base `develop` ← compare `feature/postgres-docker`. Revisar y hacer merge.
- **PR 2:** base `main` ← compare `develop`. Revisar y hacer merge.
- **No borres** la rama `feature/postgres-docker` después del merge (ignora el botón "Delete branch").

Para actualizar tu copia local después de los merges:

```
git checkout develop
git pull origin develop
git checkout main
git pull origin main
```

10. Comandos del día a día

Comando	Qué hace
docker compose up -d	Enciende Postgres (en segundo plano)
docker compose stop	Apaga Postgres sin borrar nada
docker compose down	Elimina el contenedor; los datos siguen en el volumen
docker compose down -v	Elimina contenedor **y datos** (base limpia desde cero)
docker compose logs -f postgres	Muestra los logs de Postgres en vivo (Ctrl + C para salir)
docker ps	Lista los contenedores que están corriendo

11. Problemas comunes

Síntoma	Causa y solución
error during connect ... docker_engine o *Cannot connect to the Docker daemon*	Docker Desktop no está abierto. Ábrelo y espera a que diga **Engine running**.
Bind for 0.0.0.0:5432 failed: port is already allocated	Ya tienes otro Postgres usando el 5432. En docker-compose.yml cambia a `"5433:5432"` y en application.yaml usa `localhost:5433`.
password authentication failed for user "crediya"	El volumen se creó antes con otra contraseña. Ejecuta `docker compose down -v` y luego `docker compose up -d` (borra los datos).
Connection refused al arrancar la app	Postgres no está corriendo o aún no está listo. Revisa `docker ps` y espera el estado *healthy*.
Gradle dice que requiere Java 25 / *invalid source release*	Tu JAVA_HOME apunta a otra versión. Ver el comando de abajo.
Not a managed type: class java.lang.Object	Falta crear la entidad JPA (ver nota del paso 7). No es error de la base de datos.
WSL 2 installation is incomplete	Ejecuta `wsl --update` como administrador y reinicia.

Configurar JAVA_HOME a Java 25 (luego cierra y abre PowerShell)

```
$jdk = Get-ChildItem "C:\Program Files\Eclipse Adoptium" -Directory |
       Where-Object Name -like "jdk-25*" | Select-Object -First 1
[Environment]::SetEnvironmentVariable("JAVA_HOME", $jdk.FullName, "User")
```

Checklist final

- ☐ `docker ps` muestra `crediya-postgres` como *healthy*.
- ☐ Entras con `psql` a `crediya_db` sin errores.
- ☐ `.\gradlew.bat clean build` termina en **BUILD SUCCESSFUL**.
- ☐ La rama `feature/postgres-docker` está en GitHub.
- ☐ PR a develop y PR a main fusionados, sin borrar la rama feature.